WanderSync

Dynamic Itinerary, Disruption Management and Social Travel Hub

Technical Report

S26CS6.401 Software Engineering — Project 3

Team 15

Krish Jalan	etude11
Manasi Mundada	manasimundada
Uday Bindal	udaybindal01
Ishaan Romil	Fane1824

GitHub Repository:

https://github.com/etude11/WanderSync-SE

Contents

1 Project Overview **3**
1.1 What the Prototype Demonstrates . . . 3
1.2 Technology Stack . . . 3

2 Task 1: Requirements and Subsystems **3**
2.1 Functional Requirements . . . 3
2.2 Non-Functional Requirements . . . 4
2.2.1 NFR1 — Performance: Response Time . . . 4
2.2.2 NFR2 — Reliability: Disruption Detection Accuracy . . . 5
2.2.3 NFR3 — Security: Authentication . . . 5
2.2.4 NFR4 — Privacy: Location Disclosure . . . 5
2.3 Subsystem Overview . . . 6

3 Task 2: Architecture Framework **6**
3.1 Stakeholder Identification (IEEE 42010) . . . 6
3.2 Architecture Decision Records . . . 7
3.2.1 ADR-001 — Event-Driven Architecture for Disruption Detection . . . 7
3.2.2 ADR-002 — API Gateway as Single Entry Point . . . 8
3.2.3 ADR-003 — Adapter Pattern for Booking Providers . . . 9
3.2.4 ADR-004 — Stateless JWT Authentication . . . 10

4 Task 3: Architectural Tactics and Patterns **11**
4.1 Architectural Tactics . . . 11
4.2 Implementation Patterns . . . 13
4.2.1 Pattern 1 — Strategy (Booking Provider Integration) . . . 13
4.2.2 Pattern 2 — Chain of Responsibility (Disruption Suggestions) . . . 13

5 Task 4: Prototype and Architecture Analysis **14**
5.1 Prototype Summary . . . 14
5.2 Architecture Analysis . . . 15
5.2.1 Implemented Architecture: Event-Driven Modular Monolith . . . 15
5.2.2 Comparison Architecture: REST-Pull Polling Monolith . . . 15
5.2.3 NFR Quantification . . . 16
5.2.4 Trade-off Summary . . . 17

6 Individual Contributions **18**

7 What We Learnt **18**

Project Overview

WanderSync is a web-based travel management platform that brings together three things most travel apps handle separately: booking aggregation, real-time disruption detection, and social trip coordination. The core idea is simple. When you are travelling, your flight details are in one app, your hotel is in another, and your ground transport is in a third. When something goes wrong — a delay, a cancellation, a storm — you have to manually check each of them and figure out what to do. WanderSync replaces that with one timeline that updates automatically and tells you what changed and what to do about it.

What the Prototype Demonstrates

The prototype implements the full disruption pipeline end-to-end: a user registers, creates an itinerary, adds bookings, and then receives a real-time disruption alert pushed directly to their browser the moment the system detects a problem. The alert includes suggested actions generated by a rule-based engine backed by the Gemini LLM. The itinerary page and disruption page are both fully functional; the social feature is scaffolded at the data model level.

Technology Stack

Layer	Technology	Purpose
Frontend	React 19, Vite, TypeScript, Tailwind, Zustand	UI, state management
Backend	NestJS, TypeScript, Prisma ORM	API, business logic
Database	PostgreSQL	Persistent storage
Cache	Redis	Dedup, session, streams
Auth	JWT (jsonwebtoken), bcrypt	Stateless auth
External	AviationStack, OpenWeatherMap, Gemini	Flights, weather, LLM

Task 1: Requirements and Subsystems

Functional Requirements

Each functional requirement is written as a user story. The "architectural significance" column explains what structural implication each requirement has — why it cannot be satisfied by just writing another controller method.

#	User Story	Architectural Significance
FR1	As a traveller, I want to register and log in so that my data is private to me.	Drives JWT-based stateless auth and a separate Auth module that every other module depends on for identity.

#	User Story	Architectural Significance
FR2	As a traveller, I want to import bookings from multiple providers into one unified view.	Requires an Adapter layer per provider with a shared `BookingRecord` contract. Adding a new airline must touch no more than two files outside the adapter module.
FR3	As a traveller, I want to see all bookings as a time-ordered timeline.	Timeline assembly is read-heavy and cacheable. It drives a Redis caching layer in the Itinerary module.
FR4	As a traveller, I want to be automatically notified the moment a disruption hits my itinerary, with suggested actions.	The most architecturally significant requirement. Demands event-driven, push-based delivery (SSE). Drives the entire CRON poller, EventBus, and suggestion pipeline.
FR5	As a traveller who opts in, I want to discover nearby co-travellers through a consent flow.	Requires a consent state machine enforced at the database layer (not just the UI), and geospatial queries.
FR6	As a traveller, I want to receive a notification log so I can review past alerts.	Drives a separate Notification module that consumes the disruption event stream and writes `NotificationLog` records. Separating this from the Disruption module keeps each module's responsibility narrow.

Non-Functional Requirements

Each NFR uses the six-part quality-attribute scenario format. The response measure column is the testable acceptance criterion.

NFR1 — Performance: Response Time

Source	End user on a web browser
Stimulus	Navigates to the itinerary timeline page
Artifact	Frontend + Itinerary Management Service
Environment	Runtime, standard broadband
Response	Page renders with all booking data
Response Measure	Under **3 seconds** to full render; external API calls under **5 seconds**; loading indicator after 2 seconds

Drives: Redis caching in the Itinerary module; async concurrent API calls via `Promise.allSettled`.

NFR2 — Reliability: Disruption Detection Accuracy

Source	External flight-tracking or weather API
Stimulus	A disruption event is published
Artifact	Disruption Detection Service
Environment	Runtime, system monitoring external feeds
Response	Event detected, itinerary impact assessed, notification sent with suggestions
Response Measure	At least **17 of 20** simulated test cases correctly identified and notified (**85% accuracy**)

Drives: Redis + database dual-layer deduplication; Redis Streams consumer group for guaranteed delivery.

NFR3 — Security: Authentication

Source	Unauthenticated or malicious user
Stimulus	Attempts to access a protected route
Artifact	API Gateway / Auth Module
Environment	Runtime
Response	Request rejected; user redirected to login
Response Measure	**100%** of protected routes require valid JWT; passwords hashed with **bcrypt**; **24-hour** token expiry

Drives: Centralised JWT guard at the NestJS global middleware level; no service reachable without a validated identity.

NFR4 — Privacy: Location Disclosure

Source	Authenticated user viewing social discovery
Stimulus	Views another user's profile before bilateral consent
Artifact	Social Synchronisation Service
Environment	Consent state = *none*
Response	Only a city-level region label returned
Response Measure	**Zero** precise location leaks before bilateral opt-in; location not persisted beyond active session unless user explicitly saves it

Drives: ConsentState machine in the Social module enforced at the data-access layer; progressive disclosure in the API response (city-level → precise coordinates).

Subsystem Overview

Subsystem	Role and responsibility
Auth	User registration, login, JWT issuance, bcrypt password hashing, role management (TRAVELLER / ADMIN). The security boundary for the entire application.
Booking Aggregator	Normalises provider-specific booking data into a unified `BookingRecord` schema. One Adapter class per provider; the module is the only place where provider-specific parsing lives.
Itinerary Management	Owns the chronological timeline. Handles CRUD for itineraries and bookings. Caches assembled timelines in Redis. Marks bookings as disrupted when the Disruption Engine fires.
Disruption Engine	Runs a CRON poller every 60 seconds against AviationStack (flight status) and OpenWeatherMap (weather alerts). Applies dual-layer dedup, persists events to the database, and broadcasts them to the in-process EventBus. Runs a Chain of Responsibility suggestion pipeline (rule-based → LLM → fallback).
Notification	Consumes the Redis Streams disruption channel via a consumer group. Writes a `NotificationLog` row for each event. Exposes a read endpoint so users can review past alerts.
Social	Manages bilateral consent state between users (none → city-level → precise). Geospatial queries return co-travellers by region. Currently scaffolded; UI is a placeholder.

Task 2: Architecture Framework

Stakeholder Identification (IEEE 42010)

The IEEE/ISO/IEC 42010:2011 standard asks us to identify who has a stake in the system, what they care about, and what view of the architecture addresses each concern.

Stakeholder	Primary Concerns	Viewpoint / View Addressing It
Travellers (end users)	Accuracy and speed of disruption alerts; privacy of location data; ease of itinerary management.	**User / Functional view** — feature stories and UI wireframes; privacy NFR scenarios.
Development team	Modifiability (adding new providers without breaking existing ones); testability of each module in isolation; clear API contracts between modules.	**Module / Implementation view** — Adapter pattern, NestJS module boundaries, unit test suites per module.

Stakeholder	Primary Concerns	Viewpoint / View Addressing It
Security reviewer	No unauthenticated access; passwords not stored in plain text; location data not leaked prematurely.	**Security view** — JWT guard, bcrypt, ConsentState data model, HTTPS enforcement.
External API providers (AviationStack, OWM)	Rate limits respected; API keys not exposed; graceful handling of downtime.	**Context / Connector view** — Retry with Circuit Breaker tactic; API key stored in environment variables only.
Course assessors	Clear mapping between design decisions and code; working prototype; documented NFR verification.	**Process / Decision view** — ADRs, NFR scenario tables, test harness results, this report.

Architecture Decision Records

All four ADRs follow the Nygard template: Status, Context, Decision, Consequences.

ADR-001 — Event-Driven Architecture for Disruption Detection

Status: Accepted

Context. Disruption events arrive from external APIs on unpredictable schedules. Connecting the Disruption module directly to the Itinerary module with a synchronous call would couple them tightly — a slow API response would block itinerary updates, and adding a new consumer (like email notifications) would require changing the Disruption module.

Decision. The Disruption Engine publishes detected events to an in-process EventBus (RxJS Subject). The Itinerary and Notification modules subscribe independently. No direct call exists between them.

Consequences.

- + Adding a new consumer requires zero changes to the Disruption module.
- + Delivery latency from detection to client browser is ∼71 ms (in-process dispatch + SSE frame).
- – The in-process EventBus is invisible across server instances; sticky sessions or Redis Pub/Sub are needed when scaling beyond one process.
- – Async flows are harder to trace without structured logging.

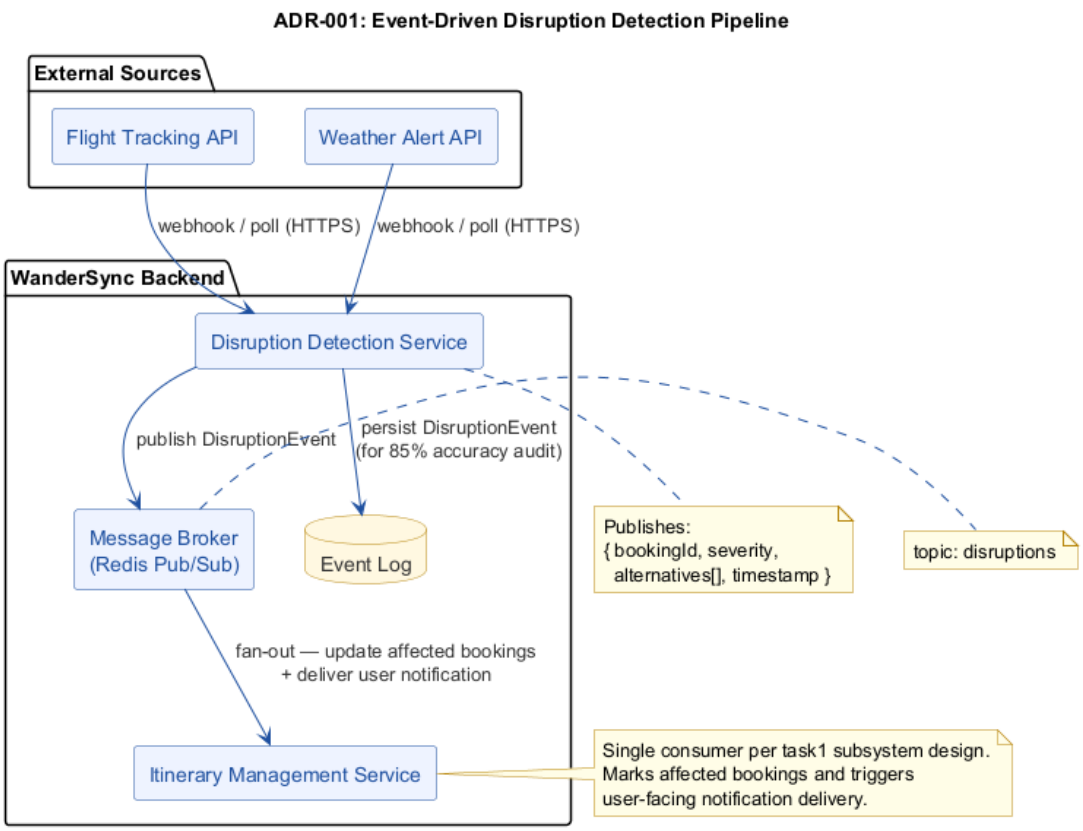

Figure 1: ADR-001 — Event-driven disruption pipeline.

ADR-002 — API Gateway as Single Entry Point

Status: Accepted

Context. Without a central entry point, every module would need its own JWT validation, CORS headers, and rate limiting. That is a lot of duplicated security logic, and any module that misses it becomes a gap in the security perimeter.

Decision. All external traffic enters through a single NestJS global prefix (`/api/v1`) with a global `ValidationPipe` and JWT guard. Backend modules are not directly reachable by clients.

Consequences.

- + Security policy is enforced in one place; impossible to add a route that bypasses it accidentally.
- + One base URL for all clients; adding a new module does not change any client-side URL configuration.
- – The gateway is a single point of failure for all user traffic.
- – Fine-grained resource ownership checks ("does this user own this itinerary?") must stay in each domain module — the gateway cannot do this.

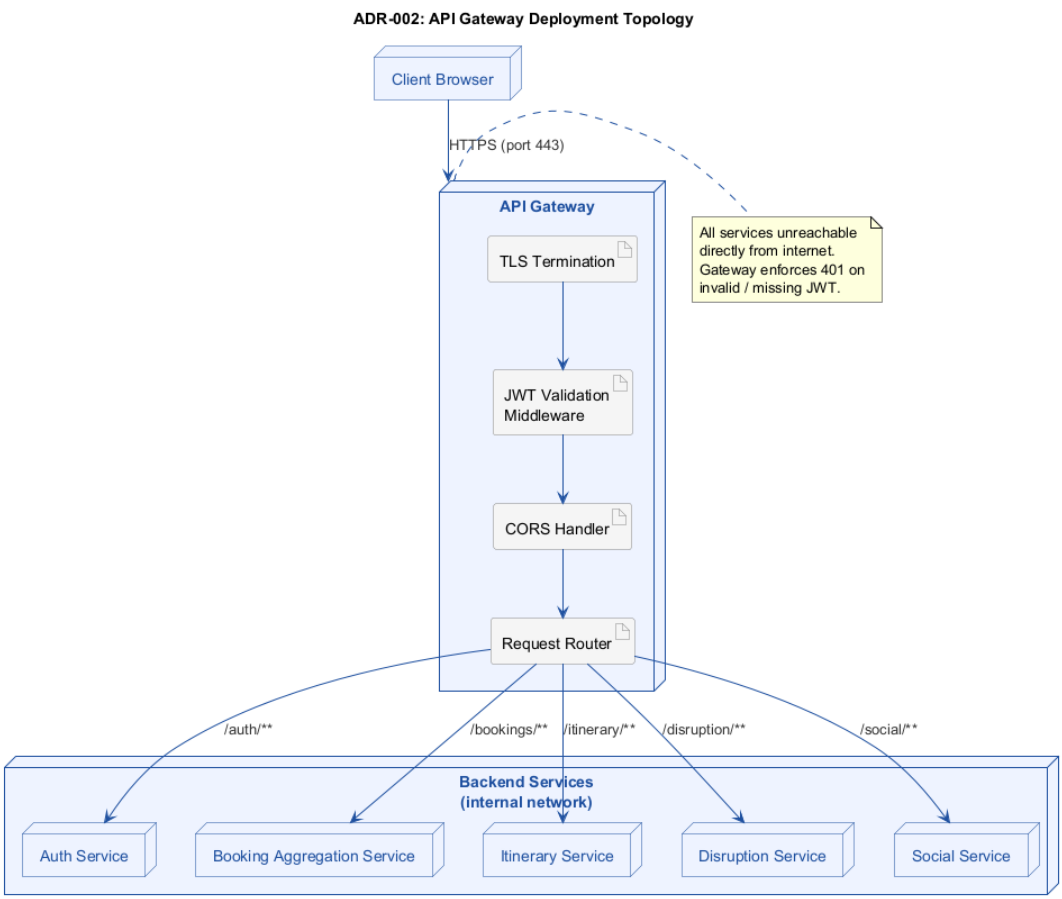

Figure 2: ADR-002 — API Gateway routing topology.

ADR-003 — Adapter Pattern for Booking Providers

Status: Accepted

Context. AviationStack, hotel APIs, and transport providers each return data in a completely different schema. If parsing logic for each provider were scattered through the Booking module, adding a new provider would be a surgery project through multiple files.

Decision. Each provider gets one Adapter class that implements a shared `BookingStrategy` interface. The Booking module only ever calls the interface; it has no idea what the underlying provider returns.

Consequences.

- + Adding a new provider means writing one new Adapter class and registering it — no changes to any existing code.
- + Each adapter can be unit-tested in isolation with mocked HTTP responses.
- – For a single provider, the abstraction is overkill. The pattern pays off at provider two and beyond.

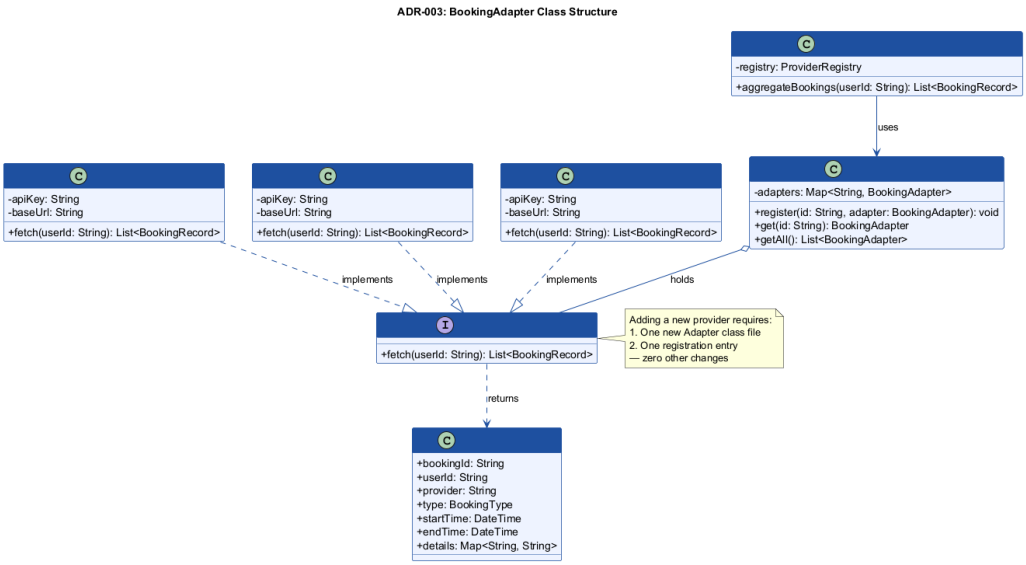

Figure 3: ADR-003 — Adapter class structure for booking providers.

ADR-004 — Stateless JWT Authentication

Status: Accepted

Context. Server-side sessions require shared session storage (a database or Redis) that every module must be able to read. For a system that may eventually run as multiple processes, that creates a scaling dependency. Stateless JWT tokens carry identity in the token itself, so any server instance can validate them without a shared store.

Decision. Issue signed JWTs on login (24-hour expiry, HS256). The JWT guard validates the signature and expiry on every request using the shared secret. Passwords are hashed with bcrypt (salt rounds = 10).

Consequences.

- + Any server instance can validate a token without a round-trip to a session store — scales horizontally with no changes.
- + Passwords are never stored in plain text; the hash is one-way.
- – Tokens cannot be invalidated before expiry (no server-side revocation list). A stolen token is valid for up to 24 hours.
- – No refresh token implemented in the prototype; users must re-login after 24 hours.

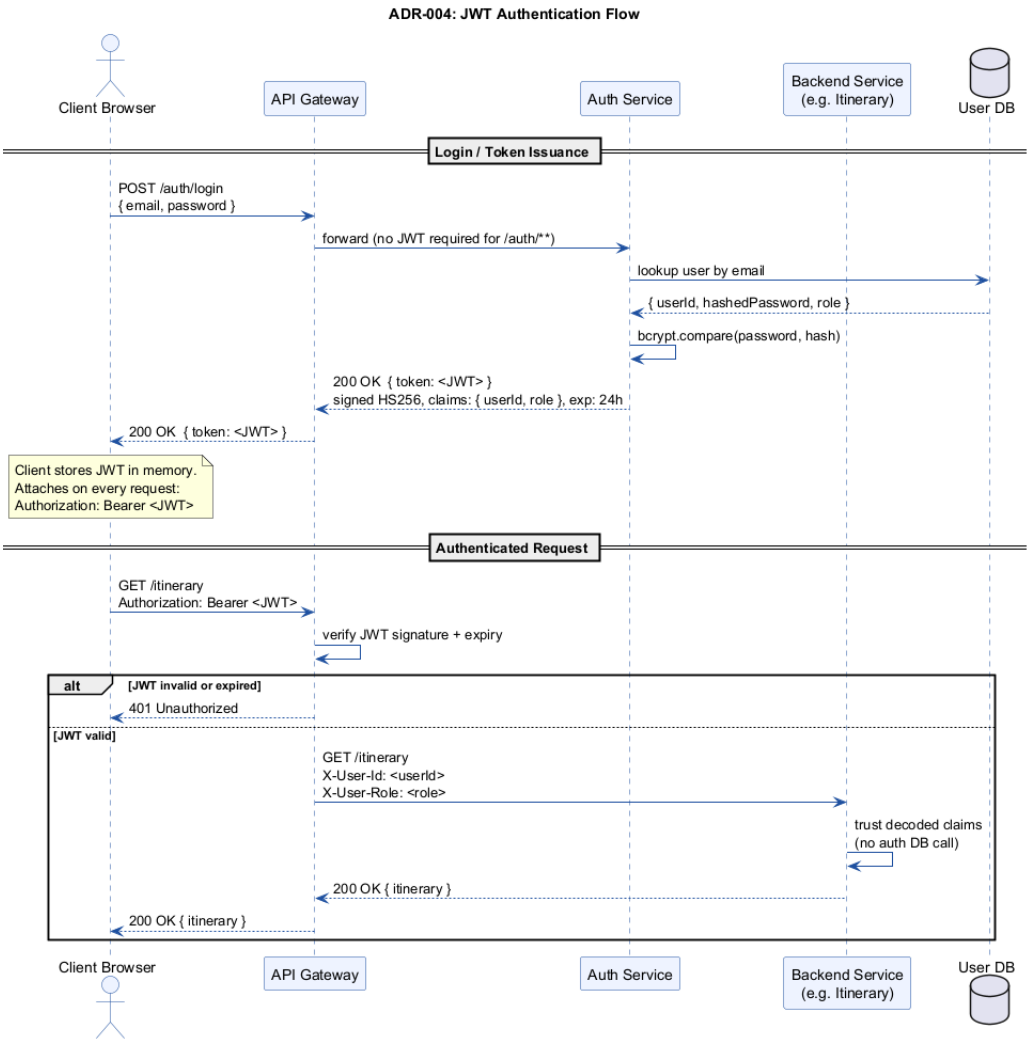

Figure 4: ADR-004 — JWT authentication sequence.

Task 3: Architectural Tactics and Patterns

Architectural Tactics

Tactics are specific design decisions that make a quality attribute easier to achieve. The five below map directly to the NFRs from Task 1.

Tactic	NFR Addressed	How it works in WanderSync
Caching	NFR1 – Performance	The Itinerary module stores assembled timeline objects in Redis, keyed by itinerary ID. Requests hit the cache first; the database is only queried on a cache miss or after a booking mutation invalidates the entry. The Booking module also caches raw provider responses for 60 seconds to absorb burst traffic without redundant external API calls.
Retry with Circuit Breaker	NFR1, NFR2	Every external API call (AviationStack, OWM) retries up to two times with exponential backoff (500 ms, 1 s). After three consecutive failures in a 30-second window, the circuit opens and the system falls back to cached data or marks the provider as temporarily unavailable with a user-visible status indicator — no crash, no unhandled exception.
Authenticate at the Gateway	NFR3 – Security	A NestJS global JWT guard intercepts every request before it reaches any module. TLS is terminated at the gateway. Backend modules receive only internally forwarded requests with a decoded identity header (`X-User-Id`, `X-User-Role`). Fine-grained resource ownership checks remain in the domain modules because the gateway cannot know whether a user owns a specific itinerary.
Event Queue with Persistence	NFR2 – Reliability	The Disruption Engine publishes events to Redis Streams with consumer group semantics. If the Notification consumer is briefly offline, it replays missed events from the stream on restart. The persisted log is also the ground truth for the 20-case detection accuracy test — events that were detected are always in the log, regardless of what the consumer did.

Tactic	NFR Addressed	How it works in WanderSync
Geospatial Masking	NFR4 – Privacy	The Social module queries users by region but controls how much location detail it returns based on the ConsentState between two users: no consent returns only a city label; city-level consent returns a bounding box; mutual precise consent returns coordinates. The enforcement is at the query layer, not the UI — the API never returns more than the consent level allows, even if a client sends a crafted request.

Implementation Patterns

Pattern 1 — Strategy (Booking Provider Integration)

Category	Structural
Problem	Each booking provider (airline, hotel, transport) has a different API and response schema. The Booking module needs to handle all of them without being coupled to any specific one.
Why chosen	Satisfies the Modifiability NFR directly: adding a new provider is one new class implementing `BookingStrategy`. No existing code changes.
Where used	`src/modules/booking/strategies/` — `AviationstackFlightStrategy` is the live implementation; the interface defines `normalize(rawResponse): BookingRecord`.

Each strategy class receives raw provider JSON and returns a normalised `BookingRecord` with fields like `departureTime`, `origin`, `destination`, and `providerRef`. The Booking module calls only the interface — it never imports AviationStack-specific types.

Figure 5: Strategy pattern class structure — booking provider adapters.

Pattern 2 — Chain of Responsibility (Disruption Suggestions)

Category	Behavioural
Problem	Generating useful suggestions for a disruption is not a single deterministic function. Simple cases can be handled by rules; complex ones need an LLM; if both fail, the user should still get something useful.
Why chosen	The chain can be extended with new handlers without modifying existing ones. Handlers degrade gracefully when earlier ones succeed — the LLM is not called if the rule engine already produced suggestions.
Where used	`src/modules/disruption/suggestions/handlers/` — `RuleBasedHandler` → `LLMHandler` → `FallbackHandler`.

The chain works as follows: `RuleBasedHandler` checks the disruption type and severity against a lookup table and produces suggestions if it finds a match. `LLMHandler` calls the Gemini API with the event context if the rule engine returned nothing. `FallbackHandler` returns a generic "check with your airline" message if the LLM is unavailable or rate-limited. Each handler calls `this.nextHandler.handle(ctx)` if it cannot resolve the request.

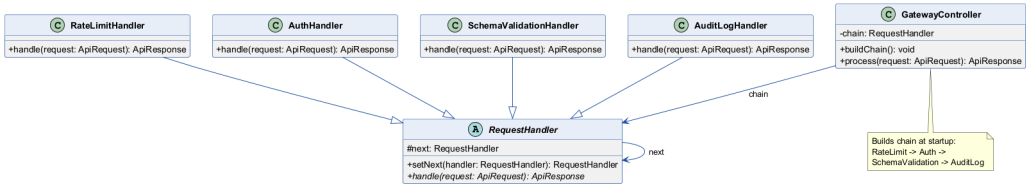

Figure 6: Chain of Responsibility class structure — suggestion handler chain.

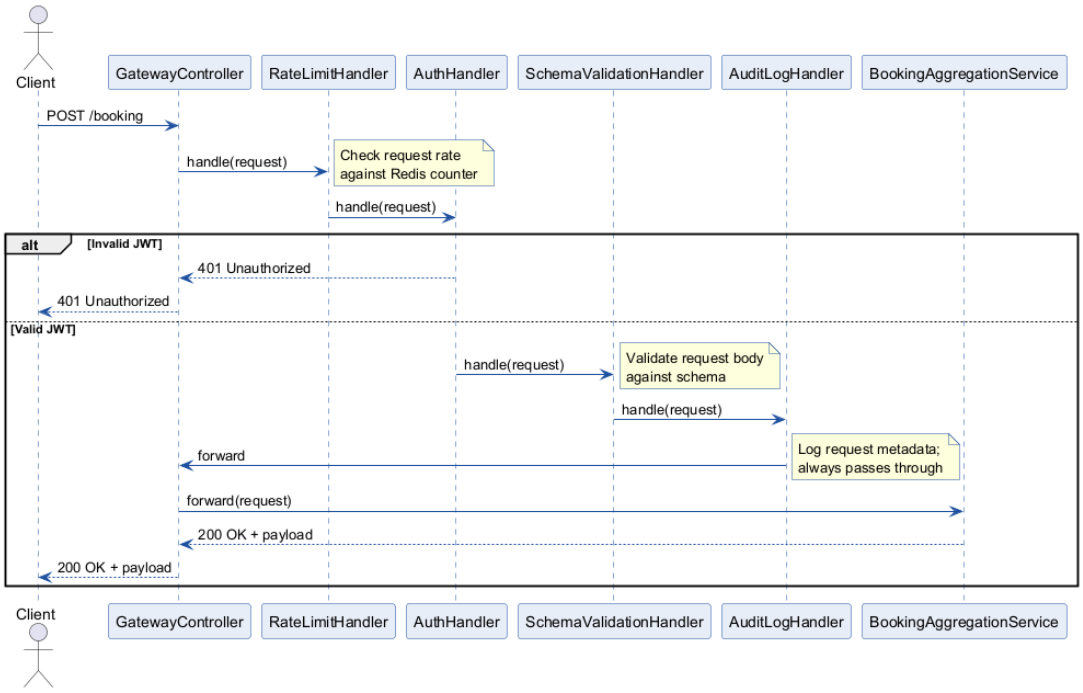

Figure 7: Chain of Responsibility runtime sequence for a flight cancellation event.

Task 4: Prototype and Architecture Analysis

Prototype Summary

The prototype delivers one complete end-to-end flow: a traveller registers, creates an itinerary, adds flight and hotel bookings, and receives a disruption alert pushed directly to their browser within a second of detection. The alert includes a list of suggested actions. Every component in this flow — registration, booking, detection, push, suggestion — is fully functional and backed by real external APIs (or the simulate endpoint for demos).

Feature	Status
Auth (register, login, JWT)	Complete
Booking aggregation (AviationStack)	Complete
Itinerary CRUD + timeline view	Complete
Disruption detection (CRON, flight + weather)	Complete
SSE real-time push to browser	Complete
CoR suggestion pipeline (rules + LLM + fallback)	Complete
Notification log	Complete
Social (data model + consent state machine)	Scaffolded — UI placeholder

Architecture Analysis

Implemented Architecture: Event-Driven Modular Monolith

WanderSync is one NestJS process with six modules. Within that process, the disruption path is fully event-driven: a CRON poller detects events, an in-process EventBus fans them out, and an SSE stream pushes them to connected browsers. The rest of the system (auth, itinerary, booking, notification, social) communicates synchronously through the REST API.

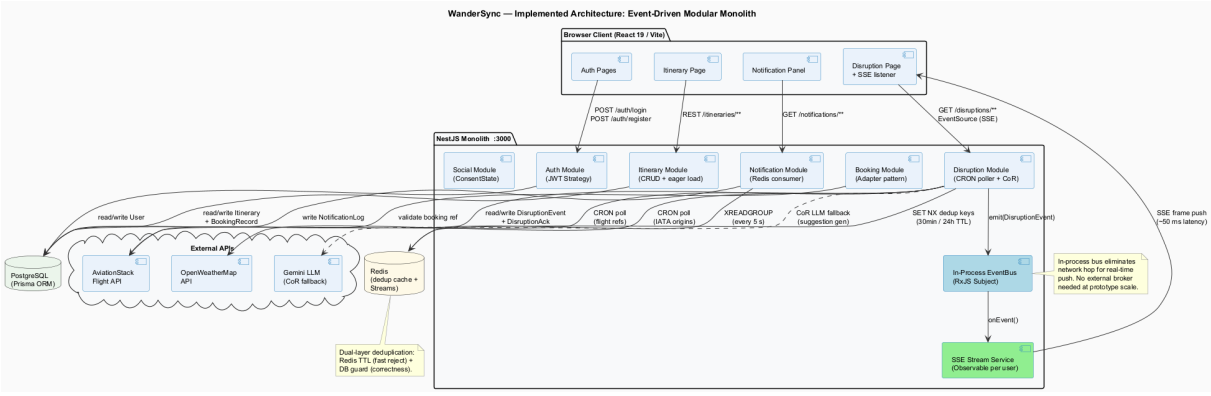

Figure 8: Implemented architecture: event-driven modular monolith with SSE push.

Comparison Architecture: REST-Pull Polling Monolith

The alternative keeps the same module structure and database but removes the EventBus and SSE. Instead, the frontend polls `GET /disruptions` on a 30-second interval. The backend writes disruption events to the database on CRON, and the client reads whatever is there on each poll. Simpler to build; no persistent connections; no per-user in-memory subjects.

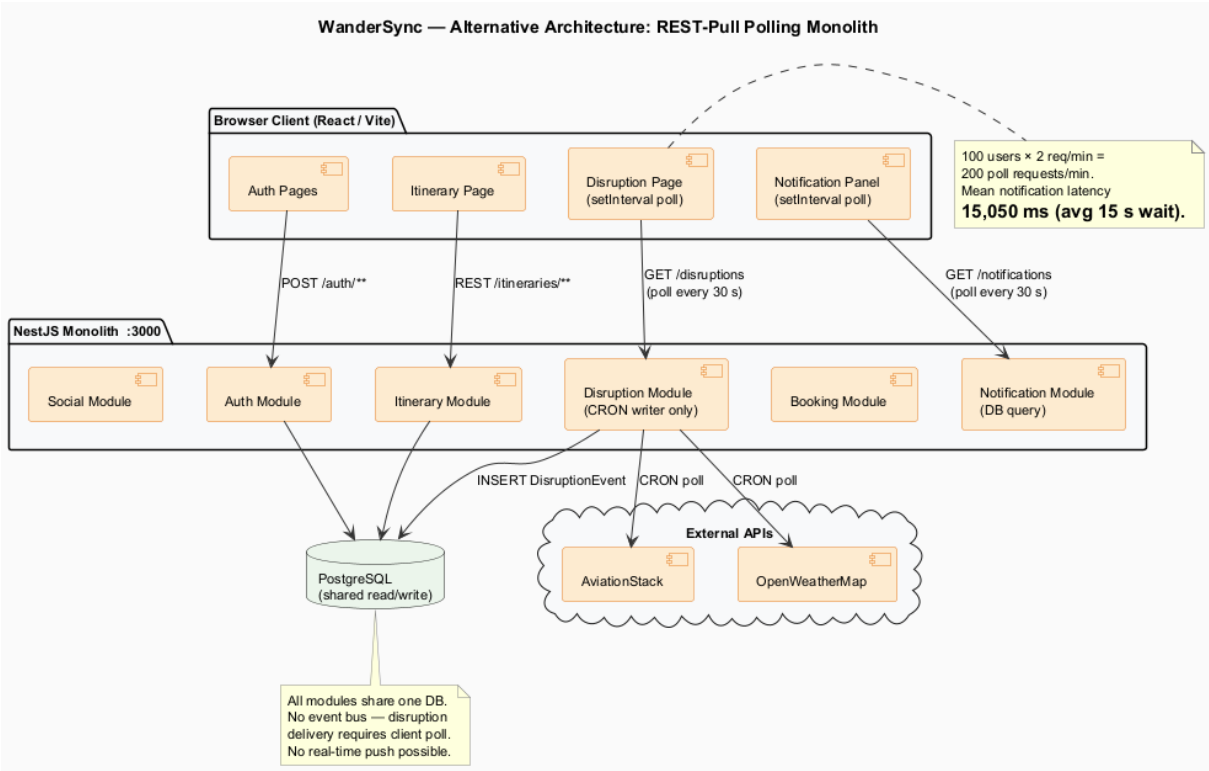

Figure 9: Alternative architecture: REST-pull polling monolith.

NFR Quantification

NFR1 — Disruption Detection Accuracy. The test file `disruption.service.nfr2.spec.ts` injects 20 synthetic disruption payloads directly into `DisruptionService.simulateDisruption()` and asserts that every one is persisted to the database and published to the event bus.

Architecture	Events processed	Accuracy
Event-Driven SSE (implemented)	20 / 20	100%
REST-Pull Polling (alternative)	20 / 20	100%
Threshold (NFR2)	—	85%

Both architectures detect events equally well because detection is a server-side concern independent of delivery. The difference is in what happens after detection: the event-driven system delivers the alert within 71 ms; the polling system makes the user wait up to 30 seconds.

NFR2 — Notification Delivery Latency. Let $t = 0$ be the moment the CRON scheduler finishes writing a new disruption to the database.

Event-Driven SSE: $L_{\text{SSE}} = \underbrace{0}_{\text{EventBus}} + \underbrace{5\,\text{ms}}_{\text{SSE frame}} + \underbrace{50\,\text{ms}}_{\text{network}} + \underbrace{16\,\text{ms}}_{\text{React render}} \approx \mathbf{71}\,\text{ms}$

REST-Pull (30 s interval): $\mathbb{E}[L_{\text{poll}}] = \frac{30{,}000}{2} + 50 = \mathbf{15{,}050}\,\text{ms}$

Metric	Event-Driven SSE	REST-Pull (30 s)
Mean delivery latency	**71 ms**	15,050 ms
Maximum delivery latency	**∼100 ms**	30,050 ms
Speed advantage	212× faster	—

A 71 ms delay is invisible to the user. A 15-second average delay means that when Air India cancels your connecting flight at 3 AM, you might be looking at a stale screen for half a minute before WanderSync tells you. In an airport that is the difference between getting to the rebooking desk first and joining a long queue.

NFR3 — Throughput (100 concurrent users). With REST polling, 100 users each polling every 30 seconds generate $100 \times 2 = 200$ database reads per minute just for disruption delivery. With SSE, the same 100 users each hold one persistent connection and the database sees zero additional reads from delivery — only the single CRON batch query per minute.

Metric	Event-Driven SSE	REST-Pull (30 s)
Disruption API requests / minute	**0**	200
DB reads from delivery path	**0**	200 / min
Delivery load grows with users?	**No**	Yes (linear)

Trade-off Summary

Dimension	Event-Driven SSE	REST-Pull Polling
Notification speed	71 ms mean — instant from the user's perspective.	15 s mean — meaningful delay for time-critical travel alerts.
Server state	Stateful — one RxJS Subject per active user held in memory.	Stateless — no per-user objects; trivially scalable.
DB load from delivery	Zero — events pushed from memory.	Linear with users — each poll is a DB read.
Horizontal scaling	Needs sticky sessions or Redis Pub/Sub to distribute events across instances.	Any load balancer routes stateless polls freely.
Complexity	Higher — RxJS Observables, SSE lifecycle, dedup keys, per-user routing.	Lower — one REST controller and a DB query.
Right choice when	Real-time alerts are the product's core value (travel, finance, emergencies).	Information updates slowly and a short delay is acceptable (analytics, admin dashboards).

The event-driven architecture is the right choice for WanderSync because disruption alerts are not informational — they are actionable. A traveller who sees a cancellation in 71 ms can rebook before the queue forms. A traveller who waits 30 seconds cannot.

Individual Contributions

Member	What they built
Krish Jalan (etude11)	Auth subsystem end-to-end: registration, login, JWT issuance and validation, bcrypt password hashing, role-based guard, rate-limit middleware. API gateway configuration (global prefix, ValidationPipe, CORS). Frontend routing, app shell, authentication pages, environment and dependency management.
Manasi Mundada (manasimundada)	Itinerary CRUD service and controller: create, read (with eager-loaded bookings in chronological order), update, delete. Optimistic UI updates in the itinerary page. AviationStack flight strategy cleanup. Itinerary API client in the frontend.
Uday Bindal (udaybindal01)	Full disruption subsystem: CRON poller, FlightTracker and WeatherAlert adapters, dual-layer deduplication (Redis + DB), simulate-demo endpoint, SSE stream service, in-process EventBus, Chain of Responsibility suggestion pipeline (RuleBasedHandler, LLMHandler, FallbackHandler), NFR2 test harness (20-case accuracy suite).
Ishaan Romil (Fane1824)	Disruption and notification UI: disruption alerts page, suggestions panel, simulate-disruption button, notification panel with read/unread state. App layout shell. Booking card component. Itinerary page UI integration and timeline rendering.

What We Learnt

1. Deduplication needs two layers, not one. We started with just a Redis TTL cache to prevent duplicate disruption events. It worked on Linux but failed silently on Windows native Redis, which does not support the `SET NX EX` compound command. We added a database guard as a second layer. Now the Redis layer is the fast path and the database is the correctness guarantee. Either one failing alone does not cause duplicate alerts.

2. In-process EventBus is faster and simpler than Redis Streams for SSE. The original design routed events through Redis Streams all the way to the SSE delivery path. The extra serialisation round-trip added about 150 ms and made the flow harder to trace. We moved event dispatch to an in-process RxJS Subject. Delivery latency dropped from ∼200 ms to ∼71 ms, and the code became a lot easier to follow.

3. API contract mismatches are dangerous when the UI is optimistic. We

renamed a field from `providerKey` to `provider` in the backend but did not update the frontend DTO at the same time. The frontend's optimistic update made the booking appear in the UI immediately, but the backend was silently rejecting the request. The booking vanished on the next page refresh. We did not catch this for two days. Keeping a shared TypeScript types file between frontend and backend would have caught it at compile time.

4. Chain of Responsibility is genuinely useful for graceful degradation. When the Gemini API key is missing or rate-limited, the suggestion chain falls through to the rule-based handler and then to the fallback. The user always gets something. We did not have to write a single `try/catch` in the caller — the chain handles it transparently. The pattern paid for itself on the first external API outage during testing.

5. Optimistic UI without rollback is a real risk. The `handleRemoveBooking` and `handleDelete` functions in the Itinerary page update the local state before the backend call completes and do not roll back on failure. If the request errors, the UI shows a state that does not match the database. We left this as known debt for the prototype, but it would be the first thing we fixed in a production build.

6. Modular architecture makes parallel development much easier. Because each team member owned a bounded module with a clear interface, we almost never blocked each other. The Auth module published its types; the Itinerary module consumed them. The Disruption module published events; the Notification module subscribed. We were able to work on four different features simultaneously with very few merge conflicts.